2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS) | 979-8-3503-3744-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ACSOS58161.2023.00020

Energy Efficient Scheduling for Serverless Systems

Michail Tsenos, Aristotelis Peri, Vana Kalogeraki
Department of Informatics
Athens University of Economics and Business, Athens, Greece
{tsemike,ariperi,vana}@aueb.gr

***Abstract*—Serverless computing, also referred to as Function-as-a-Service (FaaS), is a cloud computing model that has attracted significant attention and has been widely adopted in recent years. The serverless computing model offers an intuitive, event-based interface that makes the development and deployment of scalable cloud-based applications easier and cost-effective. An important aspect that has not been examined in these systems is their energy consumption during the application execution. One way to deal with this issue is to schedule the function invocations in an energy-efficient way. However, efficient scheduling of applications in a multi-tenant environment, like FaaS systems, poses significant challenges. The trade-off between the server's energy usage and the hosted functions' performance requirements needs to be taken into consideration. In this work, we propose an Energy Efficient Scheduler for orchestrating the execution of serverless functions so that it minimizes energy consumption while it satisfies the applications' performance demands. Our approach considers real-time performance measurements and historical data and applies a novel DVFS technique to minimize energy consumption. Our detailed experimental evaluation using realistic workloads on our local cluster illustrates the working and benefits of our approach.**

***Index Terms*—serverless, energy efficient, cloud computing, systems, scheduling**

I. Introduction

Serverless computing, also referred to as Function-as-a-Service (FaaS), has emerged as a powerful cloud computing model that has attracted significant attention and adoption in recent years. This model allows developers to write and deploy applications without the need to manage the underlying infrastructure. The Cloud providers are responsible for all the underlying infrastructure aspects such as scaling, provisioning, and maintenance. Serverless computing provides several benefits such as cost-efficiency, high elasticity, scalability and ease of use, making it an attractive option for developers looking to build and deploy applications quickly and efficiently. Some well-known commercial Serverless platforms include AWS Lambda [1], Google Cloud Functions [2], and Azure Functions [3]. These platforms simplify the development process by allowing developers to upload their application code written as a set of stateless functions, which is then packaged into containers by the platforms. For more flexibility, Serverless platforms like AWS Fargate [4] and Google Cloud Run [2] enable developers to upload their own Docker containers. In addition to commercial platforms, there are also open-source Serverless platforms available, such as OpenFaaS [5] and OpenWhisk [6]. These platforms provide the ability to businesses to host Serverless applications on their own private infrastructures, giving them greater control and customization options. As a result, FaaS has found applications in a wide range of domains. It facilitates efficient data processing, real-time analytics and handling of large datasets in the field of data processing and analytics. In the IoT domain, serverless computing enables seamless communication and real-time analytics for sensor data. Furthermore, it is widely adopted for developing chatbots [7], voice assistants, event-driven applications, and real-time applications [8] [9]. Additionally, serverless architectures are utilized in image and video processing [10], microservices and APIs, DevOps automation, e-commerce, and online retail [11]. They also support machine learning and AI tasks, including model training, inference serving, and data processing. With its versatility and flexibility, serverless computing continues to expand its applications, benefiting developers and organizations across a large variety of domains.

The applications are typically deployed as a set of stateless functions running within containers. The containers constitute a consistent and dependable method for deploying applications in real-life settings and offer isolation among the application functions. In these systems, containers from different functions share the same physical host machines and run concurrently. Consequently, scheduling decisions on the system should take into consideration the resource needs of all the hosted containers. In alternative cases, there is a chance that optimizing the performance of one container can degrade the performance of others.

One aspect of increasing concern is the amount of energy consumed by the functions when they execute on public or private cloud infrastructures. Prior work [12] has shown that the cost of powering servers housed in largescale datacenters comprises about 30% of the total cost of ownership (TCO) of modern datacenters. Furthermore, various studies report that datacenters contribute over 2% of the total US electricity usage in 2010 [13]. Although serverless systems can save energy by scaling down functions that remain idle for a prolonged amount of time to zero instances (or replicas), a significant amount of power is still needed during the function execution. Studies have shown that the consumed power by the CPU is related to CPU utilization and CPU frequency. Another aspect of concern is the performance requirements of the different functions. Typically the performance of a function is affected by the container size (CPU, memory) of the function that is chosen by the user during the function initiation procedure, or in other systems such as [14] the user can explicitly set an

979-8-3503-3744-0/23/$31.00 ©2023 IEEE
DOI 10.1109/ACSOS58161.2023.00020

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

SLA target, typically response time, for the function execution and the system automatically adjusts the replication factor of the functions in order to achieve that target.

Dynamic Voltage and Frequency Scaling (DVFS) is a power management technique used in modern processors to optimize performance while minimizing power consumption. This works by adjusting the voltage and clock frequency of the processor dynamically, depending on the workload demands at any given moment. By dynamically adjusting the voltage and frequency, *i.e.,* lowering their values when the workload is light or increasing them under heavy workload, DVFS can achieve significant energy savings without violating SLAs. DVFS has been widely adopted in mobile devices and other battery-powered systems, making it a key aspect of modern power management strategies.

However, applying DVFS in servers that host serverless functions is a challenging process due to the multi-tenant nature of those systems as described above. Each server typically hosts functions from different users that all run in parallel. Function containers can share the same CPU cores, so by adjusting the CPU frequency of the host, the performance of different functions running on the same server can be affected. Even a small decrement in the frequency can cause multiple functions to violate their SLAs.

In this work, we address the dual problem of *minimizing the energy consumption* in clusters that host serverless functions while also *meeting each function's performance requirements* (i.e. expressed via deadline constraints on their execution times). We propose a novel scheduler that can determine the placement of multiple instances of serverless function containers in a cluster of nodes. It can automatically adjust each node's CPU frequency in order to minimize the total energy consumption of the cluster while also meeting each function's performance requirements. In summary, the key contributions of the paper are:

- We formulate the problem of energy-efficient scheduling in clusters that host serverless functions. Our goal is to satisfy the deadlines and performance constraints set by the various functions and at the same time reduce the total energy consumption in the cluster, leading to the minimization of the cluster's carbon footprint.
- We propose a novel scheduling algorithm that identifies the best placement of the serverless function containers in the cluster based on various metrics or historic data derived from the functions, the provided function SLAs, and also the type of workload. It can also automatically apply DVFS techniques to the cluster hosts in order to reduce the power footprint of the cluster.
- Finally, we provide an extended experimental evaluation of our Scheduling algorithm in our local cluster. Our experimental results illustrate that our approach is practical, can effectively schedule different types of functions in the cluster and reduces the total amount of consumed energy in the cluster.

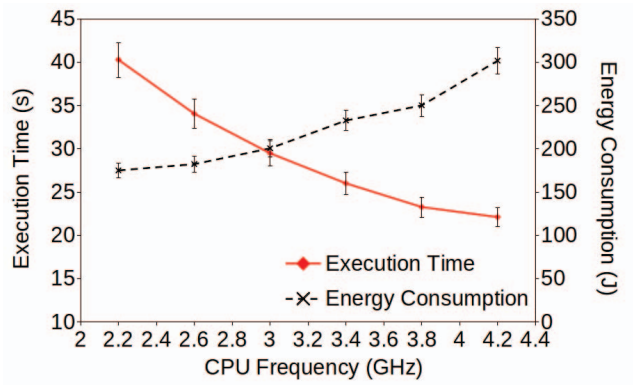

Fig. 1. Execution Time vs Energy Consumption for a benchmarking workload

II. Motivation

As mentioned above, modern CPUs enable the dynamic selection of their clock frequencies in order to save power during low-load conditions. Typically the operating system of the host machine is responsible to adjust that frequency through some pre-defined energy-saving policies. When running a CPU-intensive application, the application's performance is directly affected by the CPU frequency. If the CPU frequency is high, the application will be able to execute more instructions per second, leading to faster performance. On the other hand, if the CPU frequency is low, the application will be able to execute fewer instructions per second, leading to slower performance. Users expect different execution times from different applications. In Serverless Functions, for example, a user might expect a minimum processing throughput or a maximum response time for each request that arrives to the function. Response times of 250ms or 100ms both satisfy an SLO of the maximum response time of 300ms.

As a motivation experiment, we demonstrate the performance behaviour of a CPU-intensive function running under different CPU frequencies while we also measure the consumed energy by the CPU. For our benchmark, we wrote a function in Go that performs 10,000,000 cycles of SHA256 on a string of 50 bytes once it is triggered by an external HTTP request. We packaged that function into a Docker Container and we run 4 instances in parallel on the same host running Ubuntu 20.04 LTS with an Intel(R) Core(TM) i7-7700 with 4 physical cores and Hyperthreading disabled. We tested frequencies from 2.2GHz up to 4.2GHz. We measured the consumed energy in Joules during the test with Powercap Util. We executed four requests in parallel, one for each instance, and we measured the response time of each request. Because the requests run in parallel all the process of all the requests took the same time. Thus the average execution time matches the execution time for each request. During the execution of the requests the *CPU was constantly fully utilized*.

As we observe in figure 1, as the CPU frequency increases the execution time decreases. For example, by reducing the CPU frequency from 4.0GHz to 3.6GHz we increase the

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

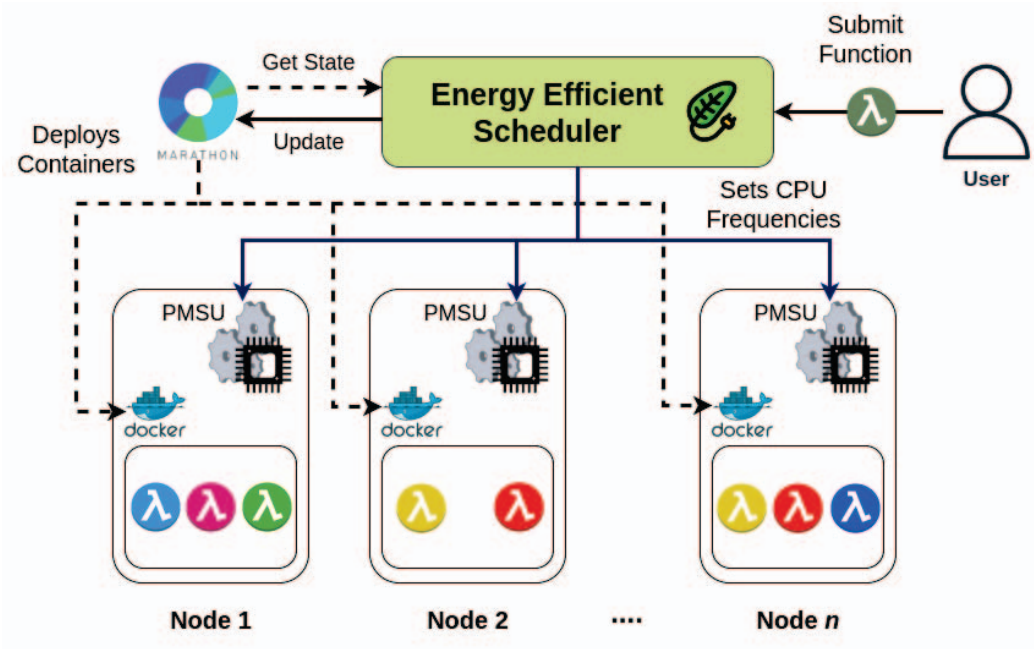

Fig. 2. System Architecture

execution time by 10%, but we also reduce the energy consumption by 22.5%. In cases where the response time is important, but not critical, this increase in latency will not affect the function SLOs. At the same time, Serverless Providers can benefit from reduced power consumption, which leads to lower operational costs and a reduced carbon footprint of the datacenter.

III. Architecture

Our system comprises two main components to operate the corresponding functionalities: (a) the Processor Management and Scheduling Utility and (b) the Energy Efficient Scheduler. Both components can be adapted in order to work in concert with any container orchestrator that allows the user to override the default scheduling policies or to explicitly select the container placement at the nodes.

A. Processor Management and Scheduling Utility (PMSU)

The Processor Management and Scheduling Utility or PMSU for short, is a small system component that is responsible for the following functions:

- Adjust the clock frequency of the host CPU
- Report the current CPU clock frequency, temperature and power values
- Dispatch and schedule in real-time the containers to specific CPU cores

More specifically, the PMSU component receives frequency update requests via a custom TCP communication protocol. Upon receiving each frequency update request it returns a CONFIRM message back, while in the case that the request fails, it returns the error message and code.

The PMSU component is also responsible and periodically measures the instant power consumption of the CPU in Watts, and also the current CPU temperature. These measurements are then reported to the central Prometheus Monitoring server for further analysis and monitoring by the cluster operators.

Finally, the PMSU component selects the CPU cores that each Docker Container uses at run-time. It intercepts the Container Creation event emitted from Docker and then by calling the Docker API, obtains the CPU resources that are allocated for that container. Once it selects a set of available CPU cores, it confines the container to a set of specific CPUs or cores. This approach resolves the Noisy Neighbor problem of multi-tenant cloud systems [15] and the functions experience a more stable and predictable performance. Periodically, the PMSU transparently changes the CPU allocation of all containers in a round-robin scheme in order to avoid the creation of hot spots on the actual CPU die.

B. Energy Efficient Scheduler

Another crucial component is the Energy Efficient Scheduler (EES). EES accepts scaling requests from the operators. Its objective is to exploit historic data, performance monitors and the provided energy-efficient scheduling policies, to determine the most appropriate set of worker nodes to allocate the function container replicas in order to reduce the total energy consumption of the entire cluster and satisfy the functions performance requirements.

The EES component interacts with the container orchestrator via its API and orders it to place the containers in the selected nodes. Our scheduler is triggered whenever new incoming requests arrive at the cluster. If the newly

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

incoming function can be scheduled, we place the container in the appropriate node and it will be scheduled based on the frequency of the corresponding node. A function cannot be scheduled either in the case that processing resources are not available at the nodes or in cases that processing resources are available but the addition of the new function may cause existing functions to miss their deadlines. In our implementation, we use Mesosphere Marathon [16] on top of Apache Mesos [17] as our container orchestrator, but it can easily adapt to any other container orchestrator that provides an API that can override the placement policies, such as for example, Kubernetes [18].

The EES stores scheduling parameters such as the function SLOs and cpu frequency demands, in the function deployment files that are stored in Marathon, by exploiting its labeling capabilities.

EES communicates with each cluster node PMSU via TCP connections, through which it can adjust the CPU clock frequency at each node. In our experimental evaluation we determined that EES is capable of sending around 1500 frequency update requests per second in our equipment.

IV. Energy Efficient Scheduler Methodology

EES accepts scaling requests from users for both batch and stream processing jobs. When submitting a batch processing job, users are required to provide a deadline (d) and the total batch size (bs) of their job. On the other hand, for stream processing jobs, users need to specify the minimum desired throughput. EES comprises two components, the Scaling Component and the Scheduling Component. In Figure 3, we present a high-level flowchart illustrating the overall process of EES and providing a comprehensive overview of the key steps involved in the system's operation, highlighting the main stages and decision points.

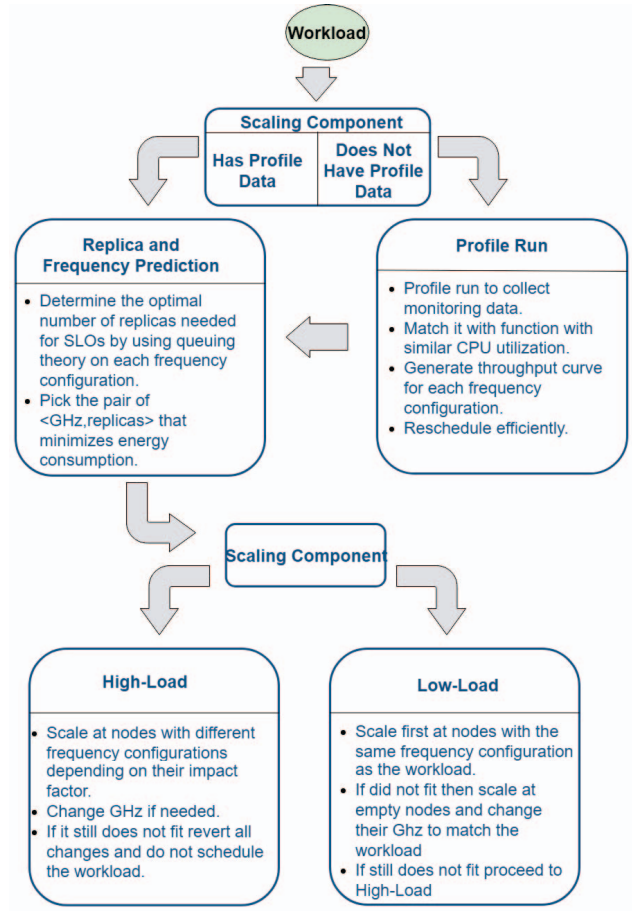

Fig. 3. Energy Efficient Scheduler Flowchart

A. Scaling Component

The Scaling Component is responsible for determining the most suitable number of instances and their frequency configuration in order to satisfy (i) the user's Service Level Objectives (SLOs), and (ii) to minimize the overall energy consumption. EESc accomplishes this by leveraging performance monitoring information obtained from historical data of previous runs of the same job. When such data exists, the Scaling Component is capable of predicting the throughput and energy consumption of a single instance for each possible GHz configuration available. We do this by utilizing a queuing theoretical model to determine the optimal number of replicas necessary to fulfil the user's desired SLOs as shown on Fig. 3 (Replica and Frequency Prediction).

Let λ denote the rate of request arrival. This rate is either directly specified by the user (in the case of stream processing jobs) or can be calculated from $\frac{bs}{d}$ (in the case of batch processing jobs). Under the assumption that the containers are homogeneous, we can conclude that all replicas c will have the same average execution time ($avgExecTime$) for the same Ghz configuration. The $avgExecTime$ can be computed by exploiting the data of previous runs. Thus, we can compute the average service rate μ for each Ghz configuration i as:

$$\mu_i = \frac{c_i}{avgExecTime_i} \tag{1}$$

Assuming a Poisson distribution with λ the rate for the incoming requests and also assuming that the services times are exponential, we can model this system using an M/M/c queuing system with c function replicas to process incoming requests in parallel. The queuing analysis of an M/M/c system is widely documented in the literature. From this analysis, the stability of the utilization factor is considered a crucial metric that needs to be met. The utilization factor, denoted as ρ, represents the percentage of time the system is occupied with jobs. To calculate ρ for each GHz configuration, the following formula is employed:

$$\rho_i = \frac{\lambda}{\mu_i} \tag{2}$$

In all queuing systems, the higher the average utilization factor, the longer the wait time for each job in the queue.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

Furthermore, when the average utilization factor exceeds 1, the queue size tends towards infinity, leading to an infinite average waiting time. As a rule of thumb, it is recommended to keep the utilization factor, denoted as r, below 80% to avoid excessive waiting times and system instability.

To determine the optimal number of replicas c_i for all possible frequency configurations in order to achieve λ, the Scaling Component solves Eq. (2) by combining it with Eq. (1). It then selects from all the available configurations P the <GHz, c_i > pair that minimizes the energy consumption $enrgCost_i * c_i$. The $enrgCost_i$ from each configuration is also obtained from historic data. The system then proceeds, by evaluating all possible configurations, to selecting the <GHz, c_i > pair that minimizes the energy consumption:

$$\arg\min_{i \in P}(enrgCost_i * c_i) \tag{3}$$

The $enrgCost_i$ value for each configuration is also constructed from historical data.

Finally, if such information does not exist then the Scaling Component deploys the job with a random pair of <GHz,1> and deploys it until it obtains some monitoring data Fig. 3 (Profile Run). The monitoring data gathered from that profile run is the percentage of the CPU utilized and the throughput of the function for that specific frequency. EES matches it with the function with the closest CPU utilization. As we observed, functions with similar CPU utilization have similar energy consumption. The throughput will follow a similar curve to the throughput curve of the similar function, displaced in the Y-axis by

$$r = \frac{throughput_{gathered}}{throughput_{known}}$$

In Fig. 4 we demonstrate how EES, given a known function, can estimate the throughput on each frequency configuration of a function with an unknown profile but with *similar CPU utilization* with a low deviation from the actual throughput. In this figure, we used 'Sha256 1' from Table III to estimate the throughput of 'Sha256 3' which has different CPU allocation and different input size. As we can see, EES predicted the throughput with high accuracy.

For jobs without historic data meeting the SLOs are not guaranteed while the actual profiling is done by the EES and for this reason, we propose the user to perform a small profile run before submitting the actual workload.

B. Scheduling Component:

The Scheduling Component is responsible for deploying jobs within the cluster. It follows the early binding scheduling model which means that jobs are immediately deployed for execution on the most appropriate available node. This approach is preferred because it can provide reduced startup latency which is crucial for real-time serverless applications with strict deadlines that have to be met. However, it is important to note that early binding scheduling does come with some drawbacks. One disadvantage is that there is no queuing stage, which means the system cannot reorder function scheduling

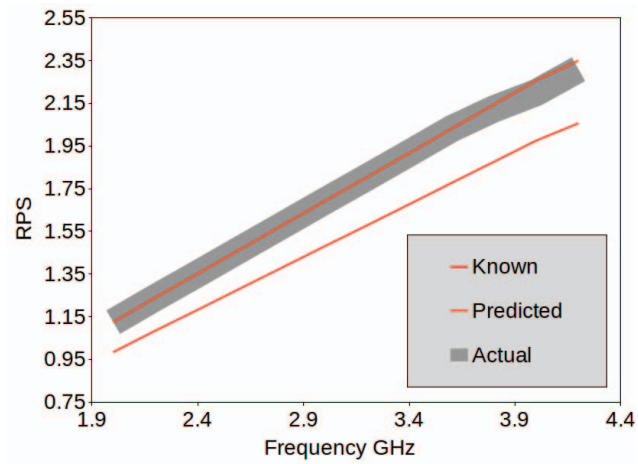

Fig. 4. Predicting an unknown function throughput (in Request Per Second - RPS) based on a known function with similar CPU utilization

to achieve more optimal allocations on the nodes. Despite its drawbacks in that regard, we believe that this trade-off is acceptable for the FaaS model. For our scheduling, we propose a straightforward but quite effective hybrid greedy load balancing scheduling model which changes depending on the load.

Low-Load: During periods of low load i.e. when there are nodes with no running tasks or there are nodes with running tasks but whose set frequency matches the job's frequency the Scheduling Component deploys them using the following greedy approach Fig. 3 (Low-Load). First, it starts from the nodes whose frequency matches the job's frequency. It then proceeds to assign function instances to available cores on these nodes, starting with the nodes that have the fewest available cores. If there are no more cores available then it picks arbitrarily from the empty nodes and after setting their frequency to match the function's frequency it starts placing replicas to them. If there are no more available empty machines the algorithm proceeds to the high-load load balancing model.

High-Load: During periods of high load i.e. when all nodes are hosting jobs and the nodes with available cores do not match the job's frequency, the Scheduling Component employs a different greedy approach for scheduling Fig. 3 (High-Load). For each node with available cores, our algorithm assesses the impact of deploying the job there. If the machine is currently running at a higher frequency than the job's frequency, deploying the job there would result in it running at a higher frequency than the one that we have identified as the most appropriate. The impact in this case would be the increased energy cost we would incur. On the other hand, if the machine is running at a lower frequency, the Scheduling Component would have to increase the node's frequency, where now the impact is the increased energy cost of the functions already running on that node, compared to their previous lower frequency. It then sorts the nodes depending

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

on that impact factor and starts deploying replicas at the nodes. If there are insufficient available cores prior to job scheduling, the job will not be scheduled, and the user will be notified accordingly. Although it is possible to deploy multiple jobs in the same core we opted against it because it causes unpredictable results for the job.

Following this greedy approach, we (i) maximize the probability that all jobs will meet their SLOs, and (ii) in cases of high-load we try to lower the energy impact where the optimal frequency for maximum energy reduction cannot be guaranteed. We achieve these goals without resorting to rescheduling, which could lead to better scheduling but at a cost of higher job delays and unpredictable outcomes.

Finally, when a job is completed, it is removed from the nodes. If the finished job had the highest frequency among the running jobs, the Scheduling Component adjusts the frequency to the second highest. In the case there are no other jobs running, it sets the frequency to the lowest available option.

V. Implementation

We have implemented our techniques in a prototype system in order to experimentally evaluate the working and benefits of our approach. For our serverless system, we use OpenFaas Function templates for building our application functions as Docker Images. We use Mesosphere Marathon 1.5 on top of Apache Mesos 1.9 as our container Orchestrator. A high-throughput HTTP reverse proxy written in Java acts as the Gateway to the function containers. It listens to container start / stop events received from Marathon and then it can automatically proxy the incoming function requests to the available function replicas. It also logs execution statistics for each function and reports them to Prometheus through the Prometheus Java client.

We have also implemented the Energy Efficient Scheduler (EES) which includes all the described scheduling policies. The Scheduler is written in Java and uses the Marathon API for deploying containers to our cluster nodes. It overrides the default scheduling policies of Marathon by creating custom application deployments which exploit node placement constraints. In this way, it can instantiate an arbitrary number of function replicas across all cluster available nodes.

The PMSU is written in Java and exploits the Docker Http API in order to interact with the Docker Engine for setting the CPU affinity (cpuset) of the running containers. It receives Frequency Update requests from the EES and uses the Debian package cpupower for setting the CPU frequency of the node. It also uses the Debian package Rapl in order to get the current CPU energy consumption and lm-sensors for monitoring the current CPU temperature. It uses the Java Prometheus client to report in real-time the power and temperature measurements.

For the Monitoring service, we use Prometheus [19] and Grafana [20] for generating real-time graphs for the various parameters that we monitor. Prometheus is a time-series database with proven performance and querying capabilities.

VI. Evaluation

In this section we present the setup and methodology we used in our experimental evaluation.

A. Experimental Setup

We evaluated our Energy Efficient Scheduling techniques in our local computer cluster which comprises twelve nodes in total. Seven of these nodes are running Ubuntu 20.04 LTS with Linux Kernel 5.15 where each node is equipped with an Intel(R) Core(TM) i7-7700 CPU with 4 physical cores, base frequency at 3.60 GHz, Max Turbo Frequency at 4.2 GHz and a TDP of 65 Watts. Each of these nodes has 16 GBs of RAM. Thus, we had 28 physical cores for deploying and executing the application functions. Similarly to [21], we deactivated Hyper-Threading to obtain more predictable results. In addition, we used five separate nodes for running the EES, the Mesos and Marathon Master and Zookeeper, the Prometheus and Grafana services, the Gateway and the load generator that is used for injecting the user workloads. All nodes are interconnected through a 1 Gbps network.

B. Evaluation Methodology

In order to provide a detailed experimental evaluation of our proposed methods, we designed and conducted the following set of experiments. Our aim in the experiments was to provide insights into how the following aspects of a serverless FaaS system are affected by our proposed methodology.

- Energy consumption
- Cold-start
- Performance

We evaluated our EES scheduler against three other techniques. The first technique that we compared is the Baseline Performance (BP). In this technique, we used the default scheduling policies of Mesosphere Marathon and Apache Mesos for the container placement of the functions, and the Linux Performance Power governor in the cluster nodes for power management. For the second technique, Baseline Powersave (BPS), we used again the same scheduling policies from Marathon, but we used the Powersave power governor of Linux in the cluster nodes. The Powersave governor sets the CPU clock frequency to the lowest setting. In our case, 2.0GHz was set as the lowest, however during the experiments we observed that the frequency jumped to over 3.0GHZ which implies that either the Kernel is not fully compatible with the CPU or the CPU overrides this setting on its own. Finally, we compared our policy against BP, but this time we used cpuset parameter of the Docker containers in order to confine the execution of each function container to a set of specified cores of the processor as described in the section III-A. This technique is defined as BP+CPU in the experimental figures.

For the three Baseline approaches we implemented a *Request Per Second* auto-scaler which is a standard scaling technique used in many serverless systems such as OpenFaas and KNative [22]. The auto-scaler periodically observes the

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
WORKLOADS DESCRIPTION

Function	Language	CPU	Network	Description
Cars	Python	High	High	Cars detection in a given image
Sha256	Go	High	Low	Performs N loops of SHA256 over a given string
Linpack	Python	Very High	Low	Solves a dense system of linear equations of size *N*
Pdf	Python	Low	Medium	Generates a PDF file by combining text with image

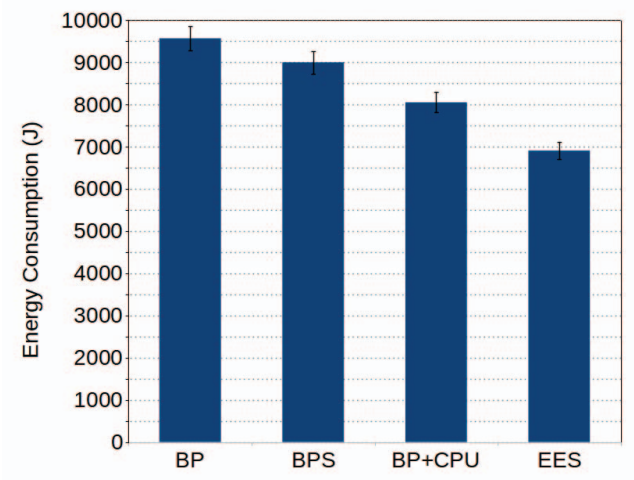

Fig. 5. Total energy consumption in Joules

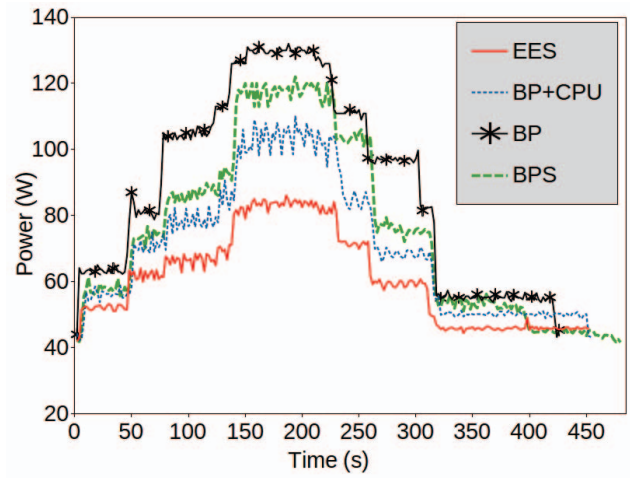

Fig. 6. Cluster power consumption at runtime with the different techniques

TABLE II
WORKLOAD ALLOCATIONS

Workload	BP replicas	BPS replicas	BP+CPU replicas	EES replicas	Baselines Freq. Range configuration	EES Freq. configuration
Car Detection	4 replicas	4 replicas	3 replicas	3 replicas	2.0-3.6 GHz	2.8 GHz
Sha256 1	2 replicas	2 replicas	2 replicas	2 replicas	2.0-3.6 GHz	3 GHz
Sha256 2	3 replicas	2 replicas	3 replicas	2 replicas	2.0-3.6 GHz	3 GHz
Sha256 3*	1 replica	1 replica	1 replica	1 replica	2.0-3.6 GHz	3 GHz
Linpack 1	1 replica	1 replica	1 replica	1 replica	2.0-3.6 GHz	2.8 GHz
Linpack 2*	6 replicas	6 replicas	1 replica	1 replica	2.0-3.6 GHz	2.8 GHz
Pdf Generation	3 replicas	3 replicas	3 replicas	3 replicas	2.0-3.6 GHz	2.4 GHz

incoming traffic for each function and calculates the required replicas as:

$$replicas = ready\ replicas \cdot \frac{mean\ load\ per\ replica}{target\ load\ per\ replica}$$

For our EES scheduler, we calculated the replicas according to the method in Section IV-A. The EES available configurations start from 3.6Ghz and can go down to 2GHz in fixed intervals of 200MHz

In parallel, we tested how the CPU frequency affects the cold start of the functions. We tested the 4 different functions with the minimum and maximum CPU frequencies that our scheduler uses.

C. Workloads

For our workloads, we used two standard benchmarks with high CPU demands (Sha256, Linpack) [23], one with high I/O writes in the storage system Minio [24] (Pdf) and one with high CPU demand and high I/O read writes to Minio (Cars). The functions used as benchmarks are shown in the following Table I. *Sha256* and *Linpack* are the two benchmark functions that aim to load the CPU to its limit. Sha256 performs N loops of SHA256 cryptographic hash function over a given string. The number N is given as a parameter at the request. For our experiments, the N value is 1000000. Linpack is a widely used benchmark workload, also used by Top500 [25], that solves a dense system of linear equations of size N. Similarly with SHA256 that N is given as a parameter at the request. We created two Linpack workloads (i) a light one where N is equal to 100 and (ii) a large one with N equal to 1000. *Pdf* is a high throughput function that generates a Pdf document by combining text and an image which are read from Minio. Finally, *Cars*, uses the YOLOv5s [26] object detection model to find the cars on an image that is read from Minio and then it stores the position of their bounding box as a text file back to it.

We deployed these functions with different CPU allocations and parameters in our cluster as shown in Table III, these were injected simultaneously in the cluster. For each deployment,

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

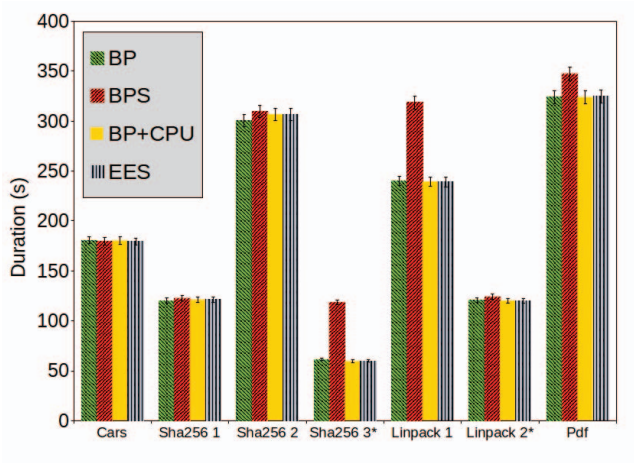

Fig. 7. Comparison of EES with State of the Art techniques in terms of Workload Duration for different workloads

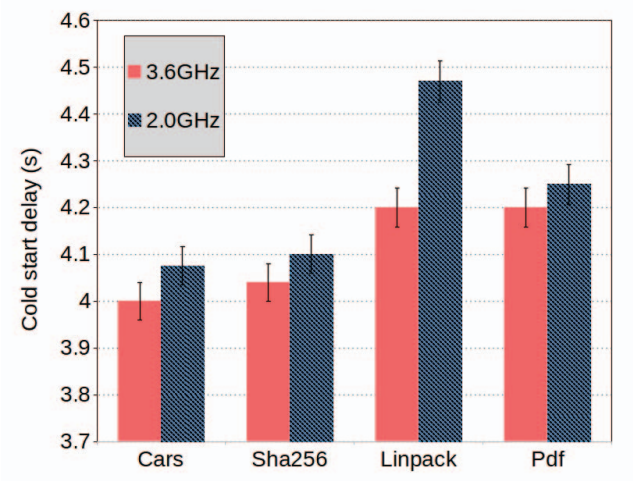

Fig. 8. Cold-start delay with minimum and maximum frequency configuration for different workloads

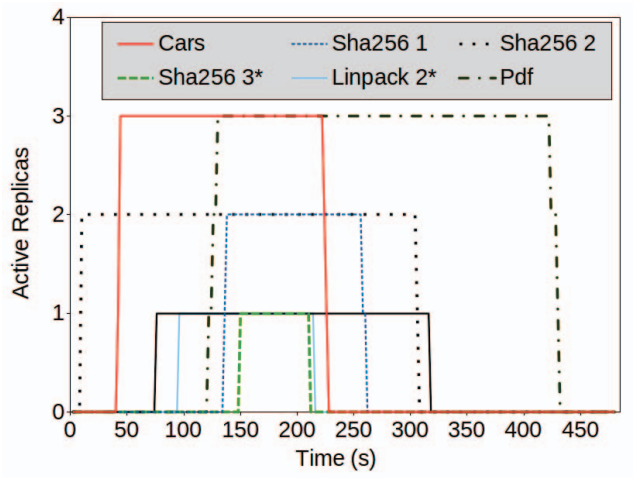

Fig. 9. Active Replicas For Each Workload with EES

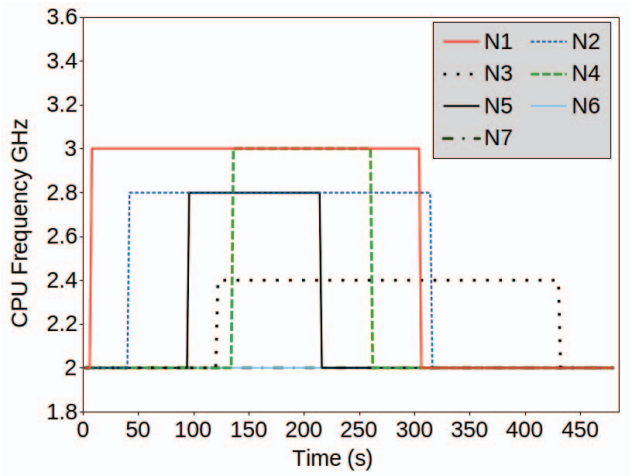

Fig. 10. Frequency selection for each node by EES

we selected a request rate from Microsoft's Azure traces [3], [27], [28] and we used Hey [29] to generate traffic. The inter-arrival time between the workloads is given by a Poisson distribution with $\lambda = 0.05$ and a fixed random seed for repeatability across the experiments. For the workloads that are indicated with "*" in their name, we did not have profile runs and used the appropriate technique that is described in section IV-A. For the rest of the workloads, we used profiles acquired from historic data.

VII. Evaluation Results

A. Energy consumption

The main goal of our EES scheduler is to minimize the total power consumption while maintaining the required performance. From Figure 5 we observe that EES is from 14% up to 28% superior to its competitors in terms of energy savings. In particular, EES achieves 28% energy savings compared to the BP approach, which is the standard frequency setting and scheduling technique used in cluster nodes. EES has significant benefits both in terms of energy savings and performance over BPS, the default powersaving mode of the baseline technique (as we also discuss later in Figure 7). In Figure 6 we depict the cluster power consumption at runtime with the different techniques. As the figure shows, the lowest power consumption was achieved with EES at all times, compared to all other methods. Note though, that, Table II illustrates that EES and BP+CPU deployed fewer replicas compared to BP and BPS for the same request rate which leads to lower total energy consumption compared to BP and BPS. Furthermore, due to the lower frequency configuration, EES achieves even lower energy consumption compared to BP+CPU. When using the CPUs in low power, the datacenters can benefit in more

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

ways that the direct reduced energy consumption. For example, by keeping the CPUs in low power, the datacenter consumes less energy for operating cooling units, because the CPUs run cooler.

B. Performance

As we observe in Fig. 7 our EES scheduler achieved the same or better performance compared to the three other techniques. This denotes that our methodology achieves its dual objective, *i.e.,* minimize the cluster energy consumption while meeting the performance goals (*i..e,* SLOs) of the scheduled functions. We also observe that the Linux Powersave governor leads to unpredictable behaviour in the majority of the workloads.

C. Cold-start

In terms of cold-start delay, in Fig. 8 we can observe that the average cold-start delay slightly increased with the lower CPU frequency with the exception of Linpack which increased by 260ms. We have to note here, that, our nodes use HDD drives instead of SSDs or NVMe drives which reduce the cold start of the containers by multiple orders of magnitude. As a result, we can conclude that by reducing the CPU frequency, our scheduler adds minimal extra delay to the cold start of the function instances and also that as expected, higher CPU frequencies result in slightly lower cold start delays.

D. Runtime System Operation

Figures 9 and 10 illustrate the operation of our system at run-time for the different workloads. In Figure 9 we show that the different workloads require different numbers of replicas in the deployed functions. Furthermore, in Figure 10 we show the frequency selection for each node in our cluster. We observe that using our approach all nodes run at lower frequencies than the base frequency of the CPU (3.6 GHz). Furthermore, two of the nodes (N6, N7) are idle, which indicates that the EES achieved better resource utilization than all its competitors.

E. Evaluation Discussion

As we can observe from the results above, EES has important benefits as it manages to achieve very similar performance with the baseline approaches in terms of throughput, while at the same time consuming less energy due to the lower frequency configurations of the nodes' CPUs. The observed results validate our hypothesis that optimizing scheduling strategies can lead to reduced energy consumption.

Furthermore, from our experimental results we can confidently say that implementing Energy-Efficient Scheduling can maintain the workloads on the same overall performance while minimizing energy consumption. By leveraging the lower frequency configurations of the nodes' CPUs, EES effectively balances the trade-off between throughput and energy efficiency.

TABLE III
WORKLOADS RESOURCES

Workload	CPU Cores	Memory	Rate Per Second
Car Detection	1	1024 MB	3.9 Rps
Sha256 1	1	125 MB	3.2 Rps
Sha256 2	1	125 MB	3.3 Rps
Sha256 3*	0.5	125 MB	1 Rps
Linpack 1	1	1024 MB	1 Rps
Linpack 2*	0.5	1024 MB	14 Rps
Pdf Generation	0.5	256 MB	138Rps

VIII. RELATED WORK

There has been significant prior work in terms of scheduling batch and stream processing workloads [30] [31] trying to satisfy applications' performance demands. In our previous work, [32] we employed queuing theory to optimize Serverless Streaming Pipelines. Yu et al. in [33] used Reinforcement Learning (RL) to learn automatically the scheduling policies through experience in clusters running serverless functions for minimizing the average invocation execution time. Authors in [34] propose a function-level scheduler and resource manager for serverless computing. It dynamically classifies and manages function requests to minimize infrastructure costs while meeting performance requirements. In their work, Kaffes et al. [21] investigate scheduling techniques for serverless systems. They introduce a scheduler that takes into account factors such as cost, load, and locality to minimize the occurrence of cold starts, in contrast to load-based policies, while ensuring high-performance levels. All of these works focus only on efficient scheduling but they do not consider possible energy savings.

Jeong et al. in [35] propose an energy-efficient service scheduling algorithm for federated edge cloud (FEC) environments. Their approach aims to minimize the total energy consumption and reduce QoS violations. The algorithm optimizes service placement and path selection by considering actual traffic requirements, leading to improved energy efficiency and reduced service violation rates compared to existing approaches. Maroulis et al. in ExpREsS [36], [37] propose a scheduler designed to minimize energy consumption while meeting performance requirements by leveraging time-series prediction models for energy usage and execution times and applying a DVFS technique for Apache Spark batch and stream workloads. Although their technique is effective it is not applicable to serverless environments due to the multi-tenancy of the hosts where an action for energy minimization for one function can affect the performance of another since they share the same physical CPU. Other approaches such as MicroFaaS [38] exploit low-powered edge ARM-based single-board computers to run functions. Due to the low power demand, they achieve lower energy consumption. Although these approaches are interesting, they lack computing power and they cannot be used for multi-tenant systems or resource-demanding workloads.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.

IX. Conclusion

In this work we have studied the problem of energy-efficient scheduling in multi-tenant serverless cloud systems. We have advanced the state-of-the-art in several ways: (1) showed that the performance and power demand of applications executing in serverless systems expressed as a set of stateless functions can be affected by different cpu frequencies, (2) provided a Processor Management and Scheduling Utility (PMSU) component that can dynamically adjust the clock frequency of the host CPU and obtain CPU clock frequency, temperature and power values at run-time which are exploited by the scheduling component when making allocation decisions, and (3) proposed a novel Energy Efficient Scheduler, EES that determines the most appropriate set of worker nodes to allocate the function container replicas in order to reduce the total energy consumption of the entire cluster and satisfy the functions performance requirements. Our experimental results indicate a clear improvement in the system's dual objective of meeting performance goals and minimizing the cluster's energy consumption when our methodology is used, outperforming current state-of-the-art techniques.

Acknowledgement

The research work was supported by the Hellenic Foundation for Research and Innovation (HFRI) under the 3rd Call for HFRI PhD Fellowships (Fellowship Number: 6812), by the European Union through the EU ICT-48 2020 project TAILOR (No. 952215), the H2020 AutoFair project (No. 101070568) and the Horizon Europe CoDiet project (No. 101084642).

References

[1] A. Lambda, https://aws.amazon.com/lambda/.
[2] Google Cloud Run, https://cloud.google.com/run, 2023.
[3] Microsoft, https://azure.microsoft.com/en-us/services/functions/.
[4] Amazon Fargate, https://aws.amazon.com/fargate/.
[5] OpenFaas, https://www.openfaas.com/.
[6] Apache OpenWhisk, https://openwhisk.apache.org/.
[7] M. Yan, P. Castro, P. Cheng, and V. Ishakian, "Building a chatbot with serverless computing," in *Proceedings of the 1st International Workshop on Mashups of Things and APIs*, ser. MOTA '16. New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: https://doi.org/10.1145/3007203.3007217
[8] D. Tomaras, I. Boutsis, and V. Kalogeraki, "Modeling and predicting bike demand in large city situations," in *PerCom*. IEEE, 2018.
[9] D. Tomaras, M. Tsenos, and V. Kalogeraki, "Practical privacy preservation in a mobile cloud environment," in *2022 23rd IEEE International Conference on Mobile Data Management (MDM)*, 2022.
[10] F. Romero, M. Zhao, N. J. Yadwadkar, and C. Kozyrakis, "Llama: A heterogeneous & serverless framework for auto-tuning video analytics pipelines," in *Proceedings of the ACM SoCC*, 2021, p. 1–17.
[11] S. Fouladi, F. Romero, D. Iter, Q. Li, S. Chatterjee, C. Kozyrakis, M. Zaharia, and K. Winstein, "From laptop to lambda: Outsourcing everyday jobs to thousands of transient functional containers," in *USENIX ATC*, Renton, WA, Jul. 2019, pp. 475–488.
[12] L. A. Barroso, J. Clidaras, and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, Second Edition*, 2013. [Online]. Available: http://dx.doi.org/10.2200/S00516ED2V01Y201306CAC024
[13] J. Koomey *et al.*, "Growth in data center electricity use 2005 to 2010," *A report by Analytical Press, completed at the request of The New York Times*, vol. 9, no. 2011, p. 161, 2011.
[14] B. Wang, A. Ali-Eldin, and P. Shenoy, "Lass: Running latency sensitive serverless computations at the edge," in *30th ACM HPDC*, 2021.
[15] Amazon AWS, https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html.
[16] Marathon, https://mesosphere.github.io/marathon/.
[17] Apache Mesos, https://mesos.apache.org/.
[18] Kubernetes, https://kubernetes.io/.
[19] Prometheus, https://prometheus.io/.
[20] Grafana, https://grafana.com/.
[21] K. Kaffes, N. J. Yadwadkar, and C. Kozyrakis, "Hermod: Principled and practical scheduling for serverless functions," in *Proceedings of the 13th Symposium on Cloud Computing*, ser. SoCC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 289–305. [Online]. Available: https://doi.org/10.1145/3542929.3563468
[22] Knative, https://knative.dev/docs/.
[23] J. Kim and K. Lee, "Functionbench: A suite of workloads for serverless cloud function service," in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, 2019, pp. 502–504.
[24] MinIO, https://min.io/.
[25] Top500, https://www.top500.org/project/linpack/.
[26] J. Redmon *et al.*, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
[27] M. Shahrad, R. Fonseca, I. n. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*, ser. USENIX ATC'20. USA: USENIX Association, 2020.
[28] Azure, https://github.com/Azure/AzurePublicDataset.
[29] Hey, https://github.com/rakyll/hey.
[30] A. Peri, M. Tsenos, and V. Kalogeraki, "Orchestrating the execution of serverless functions in hybrid cloud," in *ACSOS*, 2023.
[31] D. Tomaras, M. Tsenos, and V. Kalogeraki, "Prediction-driven resource provisioning for serverless container runtimes," in *ACSOS*, 2023.
[32] M. Tsenos, A. Peri, and V. Kalogeraki, "Amesos: A scalable and elastic framework for latency sensitive streaming pipelines," in *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*, ser. DEBS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 103–114. [Online]. Available: https://doi.org/10.1145/3524860.3539642
[33] H. Yu, A. A. Irissappane, H. Wang, and W. J. Lloyd, "Faasrank: Learning to schedule functions in serverless platforms," in *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2021, pp. 31–40.
[34] A. Suresh, G. Somashekar, A. Varadarajan, V. R. Kakarla, H. Upadhyay, and A. Gandhi, "Ensure: Efficient scheduling and autonomous resource management in serverless environments," in *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2020, pp. 1–10.
[35] Y. Jeong, K. E. Maria, and S. Park, "An energy-efficient service scheduling algorithm in federated edge cloud," in *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, 2020, pp. 48–53.
[36] S. Maroulis, N. Zacheilas, and V. Kalogeraki, "Express: Energy efficient scheduling of mixed stream and batch processing workloads," in *2017 IEEE International Conference on Autonomic Computing (ICAC)*, 2017, pp. 27–32.
[37] S. Maroulis and N. a. Zacheilas, "A holistic energy-efficient real-time scheduler for mixed stream and batch processing workloads," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 12, pp. 2624–2635, 2019.
[38] A. Byrne, Y. Pang, A. Zou, S. Nadgowda, and A. K. Coskun, "Microfaas: Energy-efficient serverless on bare-metal single-board computers," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 754–759.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:26:53 UTC from IEEE Xplore. Restrictions apply.